MIT 8103 Advanced Database Systems: Portfolio Summary

AOWLYF Dynamics Energy Services | Balogun Kabiru | Student ID: 301922731 | MIVA Open University, MIT

GitHub repository (public): https://github.com/Olawalekaybee/Advanced-database-portfolio

Case study

AOWLYF Dynamics Energy Services is a simulated electricity distribution company serving Lagos, Kano and Ogun. Smart prepaid meters send kWh, voltage, current and a timestamp every 15 minutes. Customers hold prepaid wallets and buy tokens that load units onto their meters. Staff sign in with role-based accounts, and technicians handle faults through work orders. I used this one case study for all five portfolios. PostgreSQL holds the system of record, MongoDB Atlas holds telemetry and device events, and Neo4j Aura holds the network of customers, meters, faults and technicians.

Portfolio 1: Database Design and Modelling

I designed 13 tables: user_account, customer, staff_profile, region, tariff_band, premises, meter, reading, wallet, payment, token_purchase, fault_alert and work_order. Every table has a primary key, and foreign keys link the rest. UNIQUE, NOT NULL and CHECK rules protect the data, for example non-negative balances and a token amount that must equal units multiplied by rate [1]. I normalised a flat meter record through 1NF, 2NF and 3NF and checked BCNF. I kept the wallet (money) separate from the token purchase (units issued to a meter), and I stored the rate on each purchase so a later tariff change does not alter past receipts. I tested the schema with invalid inserts, such as an unknown region and a negative balance, and the database rejected them.

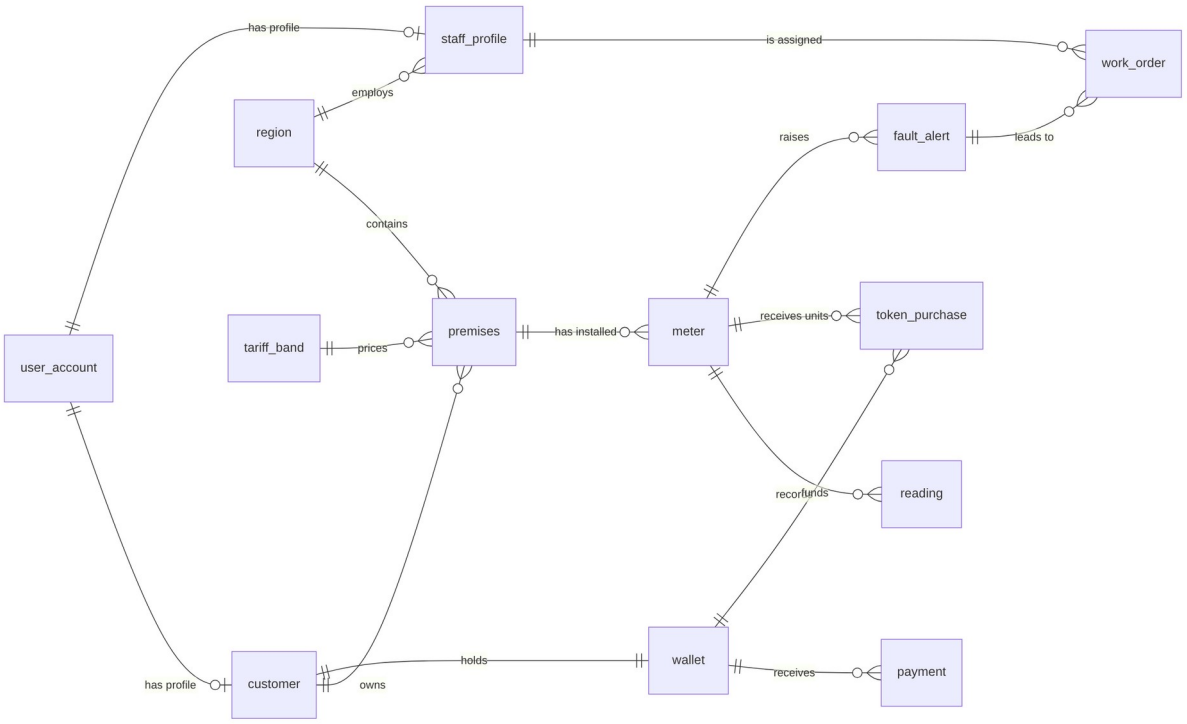


Figure 1. ER diagram of the 13 tables and their relationships. Attributes, keys and constraints are in the repository (Portfolio 1 design and schema).

Portfolio 2: Query Processing and Optimisation

I built a separate table of 500,000 readings (50 meters, one reading every 15 minutes) and compared plans with EXPLAIN (ANALYZE, BUFFERS) [2]. A composite index on (meter_id, recorded_at) turned a Parallel Seq Scan into an Index Scan. Wrapping the column in date() stopped the index from being used fully, and writing the same day as a range fixed it. I also ran three report queries on the real tables. On those few rows, switching sequential scans off made the query slower, so an index depends on data size and query shape.

Test	Plan	Execution time
No index	Parallel Seq Scan	208.864 ms
Index on (meter_id, recorded_at)	Index Scan	0.573 ms

Filter date(recorded_at) = day	Bitmap Heap Scan	4.394 ms
Same day as a range	Index Scan	0.102 ms
Report query, planner's choice (small tables)	Hash join, seq scans	0.194 ms (cost 42.13)
Same query, sequential scans off	Bitmap and index scans	0.457 ms (cost 53.31)

Each plan was run once on one laptop with cached data, so the times show a pattern, not a benchmark.

Portfolio 3: Transactions and Concurrency

A token purchase changes the wallet, the purchase table and the meter balance together. A committed purchase changed all three at once. A purchase with too little money failed the wallet CHECK, and ROLLBACK left every table unchanged. In two sessions I read the same wallet balance of 26,000.00 and sold 20,000.00 from each. The wallet fell by only 20,000.00 but two purchases were issued, which is a lost update. With SELECT ... FOR UPDATE the second session waited, then saw 6,000.00 and was rolled back, so only one purchase was issued [3]. I tested only the default READ COMMITTED level.

Portfolio 4: NoSQL and Advanced Data Models

In MongoDB, one document holds one meter's readings for one day, so a day is read in a single operation without a join. Device events with different fields share one collection. The regional totals (Lagos 6.22 kWh, Kano 0.57 kWh) match my PostgreSQL report. A compound index on meter_number and date changed the explain plan from COLLSCAN (4 documents examined) to IXSCAN (1 examined) [4]. In Neo4j I loaded 31 nodes and 32 relationships. For a faulty meter, one Cypher query finds the other meters in its region and the technicians who worked on them (Kano, AOW-KN-0005, AOW-KN-0002, Sani Abdullahi). The SQL version needs eight table references and returns the same row [5]. At this size I do not claim the graph is faster. The three stores are not synchronised automatically, and the data is a small sample.

Portfolio 5: Distributed and Cloud Database Exercise

I partitioned a copy of the readings by region_id in PostgreSQL. A Kano query scanned only the Kano partition, and a query without the region scanned all three. This runs on one server, so it shows the idea of sharding but not network failure. The rest is a proposed design, not something I built: the shard key is region_id, wallet and token purchase data would use synchronous replication and favour consistency (CP), and readings and fault alerts would use asynchronous replication and favour availability (AP). I did not set up replication, failover or a cloud deployment.

Evidence

Scripts, explanations and screenshots are in each folder: P1 design, P2 plans, P3 transactions, P4 NoSQL and P5 distributed. Portfolio 3 changes the sample wallets, so I reloaded the data before the screenshots that needed the original state.

Individual reflection

My most confident design decision was keeping wallets and token purchases as separate tables in PostgreSQL. A wallet holds money and a token purchase records the units loaded onto a meter, so the financial records stay clear and traceable. In Portfolio 3 I saw why this matters: when two transactions updated the same wallet at once, one update was lost until I used SELECT ... FOR UPDATE.

The hardest part was getting the queries and screenshots to show the right evidence. In MongoDB I first created two single-field indexes instead of one compound index, and I had to delete them and create the correct one before the plan changed to an index scan. I checked each result against what I expected and ran it again. Two results surprised me. An index cut one query from 208.864 ms to 0.573 ms, yet forcing indexes on the small tables made a report query slower. If this were a real system, I would first build a process to keep PostgreSQL, MongoDB and Neo4j updated automatically, because I load the other two stores by hand.

References

[1] PostgreSQL Global Development Group. PostgreSQL documentation: Constraints. https://www.postgresql.org/docs/current/ddl-constraints.html

[2] PostgreSQL Global Development Group. PostgreSQL documentation: Using EXPLAIN. https://www.postgresql.org/docs/current/using-explain.html

[3] PostgreSQL Global Development Group. PostgreSQL documentation: Explicit Locking and Transaction Isolation. https://www.postgresql.org/docs/current/explicit-locking.html

[4] MongoDB, Inc. MongoDB Manual: Indexes and Explain Results. https://www.mongodb.com/docs/manual/

[5] Neo4j, Inc. Cypher Manual. https://neo4j.com/docs/cypher-manual/
[6] MIVA Open University (2026). MIT 8103 Advanced Database Systems course text and lecture slides. All sources accessed November 2026. All sample data is fictional and was created for this portfolio.

AI-use declaration

I used Claude (Anthropic) to explain database concepts and to help draft and structure scripts and documentation. I ran every script myself in PostgreSQL, MongoDB Atlas and Neo4j Aura, checked the results against the assessment requirements, and corrected mistakes before submission. I remain responsible for the accuracy of the submitted work.